AUTONOMOUS DELIVERY AGENT

SUBMITTED BY :- SAMMIREDDY VARSHINI REDDY

REGISTRATION NUMBER:- 25BAI11554

SUBMITTED TO :- VIT Bhopal

ACADEMIC YEAR :- 2025-2026

INTRODUCTION

This project explores pathfinding algorithms (BFS, UCS, A*) in a grid. It models a world with static and moving obstacles, allowing an agent to plan and replan its path to a goal. It also includes tools to test and compare how efficiently these algorithms work on different maps.

PROBLEM STATEMENT

The delivery agents are not able to find the most cost efficient and time saving way to reach a particular destination.

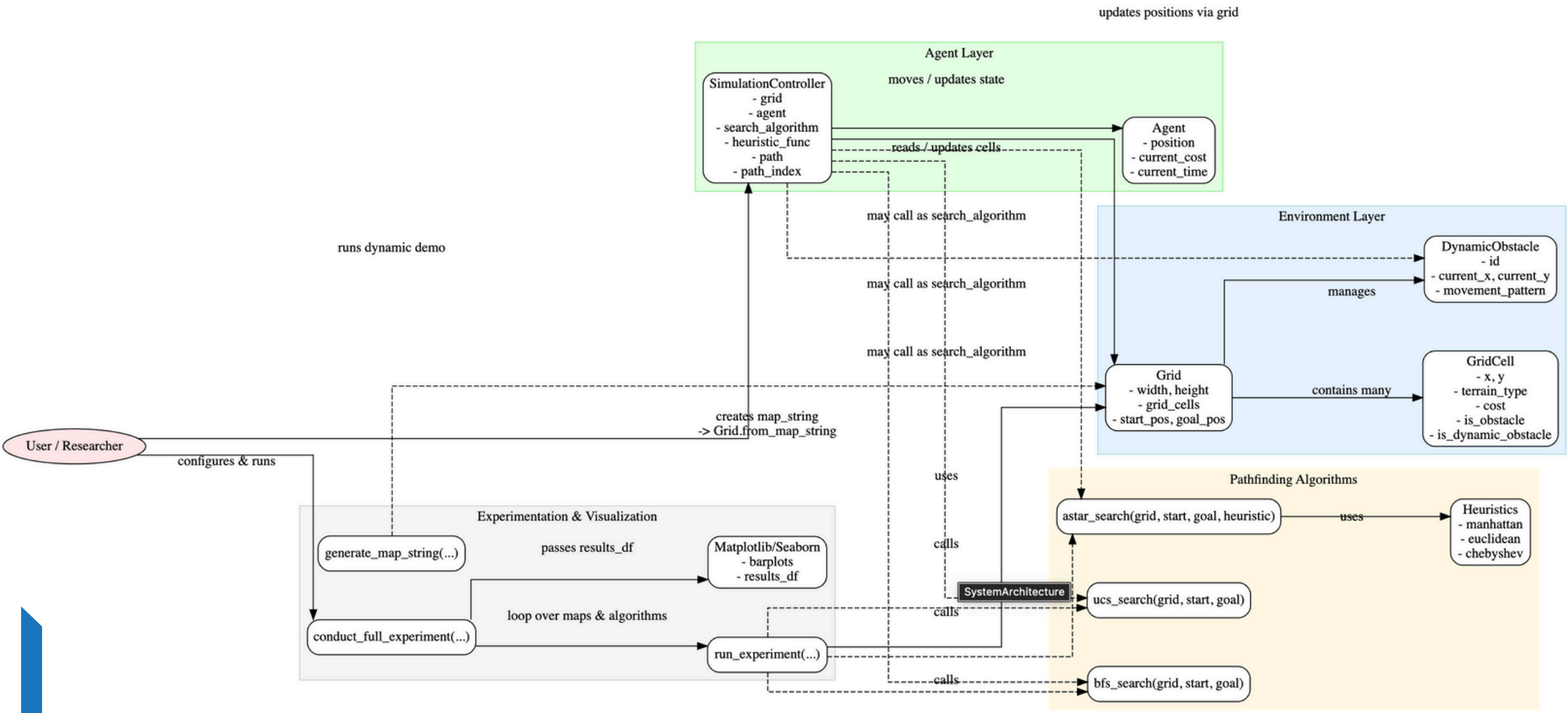
FUNCTIONAL REQUIREMENTS

- Create Maps: Build grid-based environments with different terrains and obstacles.
- Simulate Movement: Allow dynamic obstacles to move around the map.
- Find Paths: Calculate routes from a start point to a goal using different algorithms.
- Avoid Obstacles: Ensure paths don't go through static or dynamic obstacles.
- Replan Paths: Generate new paths if the current one becomes blocked by moving obstacles.
- Track Agent: Move an agent along a path, updating its position and cost.

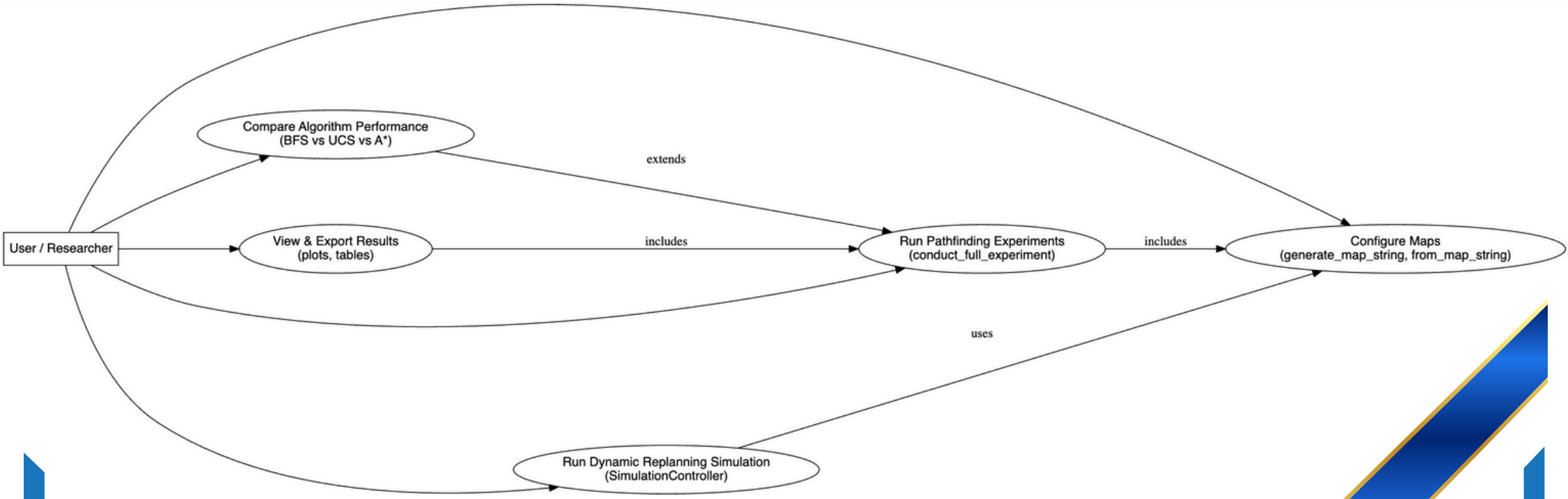
NON-FUNCTIONAL REQUIREMENTS

- Fast: Algorithms should find paths quickly.
- Efficient: Use computer resources (like memory) wisely.
- Clear: The code should be easy to understand and change.
- Reliable: The pathfinding should work correctly every time.
- Scalable: It should handle larger maps and more obstacles without breaking down too much.

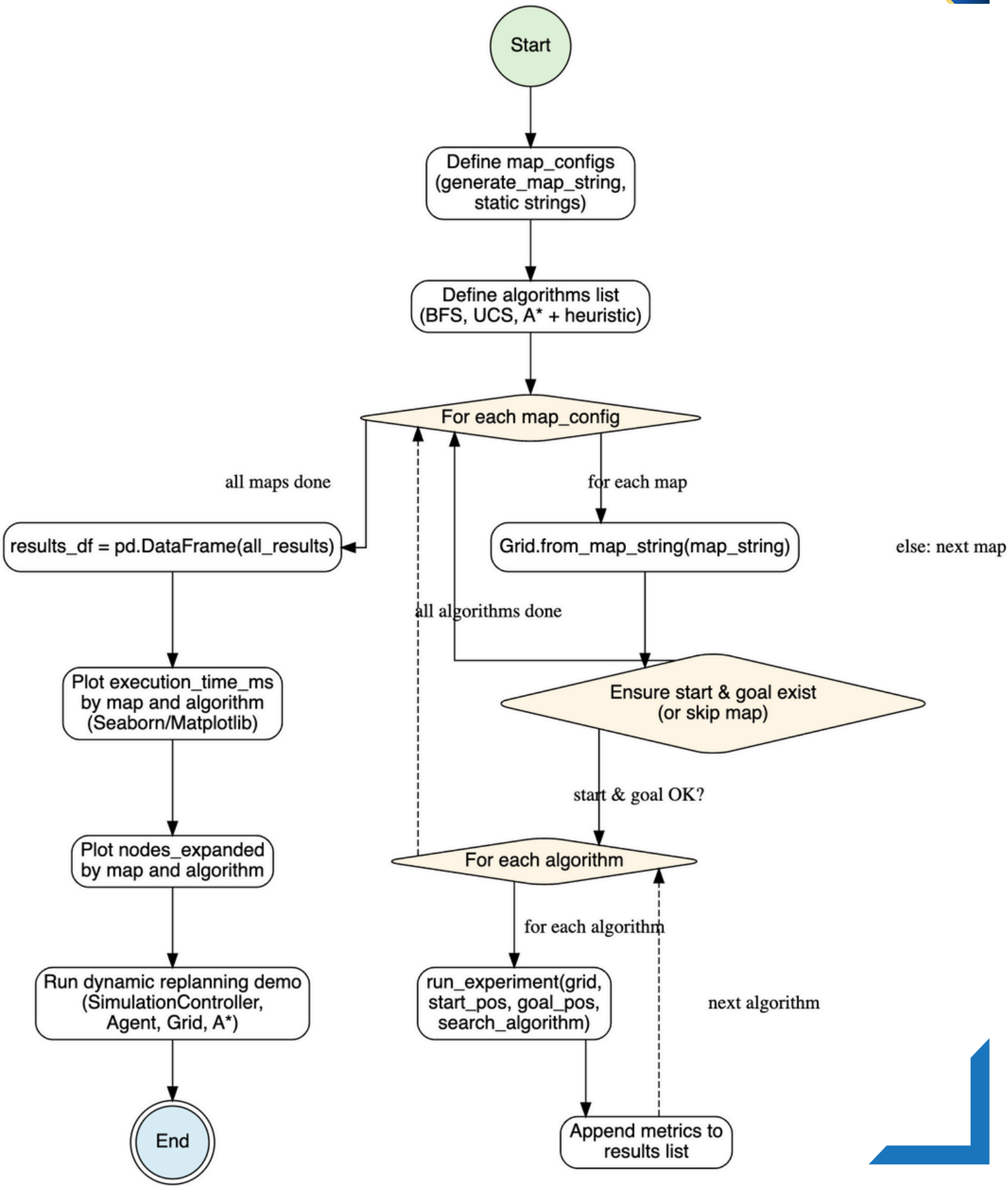
SYSTEM ARCHITECTURE



USE CASE DIAGRAM



WORKFLOW DIAGRAM



SEQUENCE DIAGRAM

DESIGN DECISIONS & RATIONALE

- Clear Parts: Separated the code into easy-to-manage sections (like Grid, Agent).
- Variety of Pathfinders: Included different search methods (BFS, UCS, A*) to compare them.
- Real-world Touch: Added moving obstacles and made the agent smart enough to react and find new ways.
- Show Proof: Measured performance to prove which methods are best for different tasks.

IMPLEMENTATION DETAILS

- Maps: Grids are made from text, each spot having a cost.
- Agent: The agent moves one step at a time.
- Pathfinding: Algorithms (BFS, UCS, A*) find the best route.
- Moving Obstacles: Obstacles move, and if they block the agent's path, the agent quickly finds a new one.

SCREENSHOTS AND RESULTS

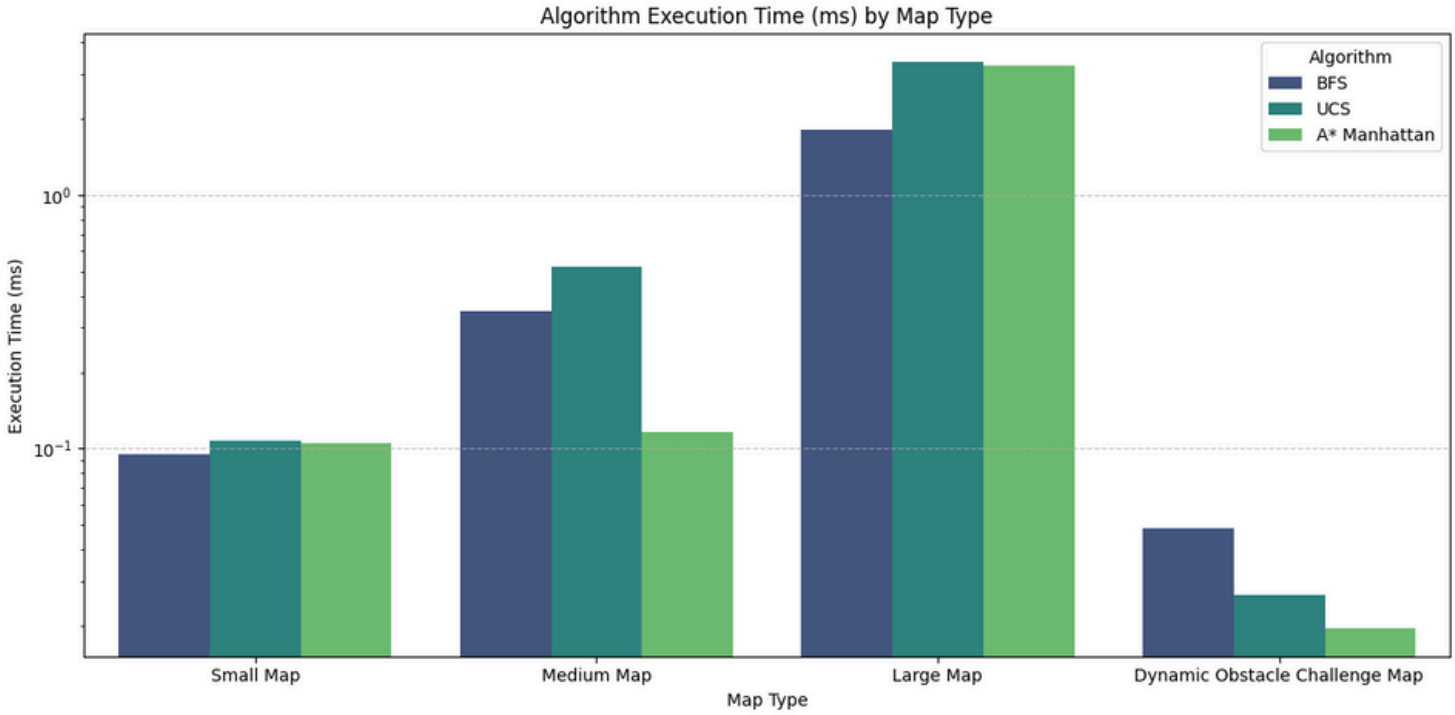
```
--- Setting Up Maps ---

Small Map:
S..#.
.....
.....
...#.
....G

Medium Map:
(Details hidden due to size, but loaded)

Large Map:
(Details hidden due to size, but loaded)

Dynamic Replanning Demo Map:

S.X
.D.
.G.


--- Conducting Full Experiment ---

Processing Map: Small Map
  Running BFS on Small Map...
  Running UCS on Small Map...
  Running A* Manhattan on Small Map...

Processing Map: Medium Map
  Running BFS on Medium Map...
  Running UCS on Medium Map...
  Running A* Manhattan on Medium Map...

Processing Map: Large Map
  Running BFS on Large Map...
  Running UCS on Large Map...
  Running A* Manhattan on Large Map...

Processing Map: Dynamic Obstacle Challenge Map
  Running BFS on Dynamic Obstacle Challenge Map...
  Running UCS on Dynamic Obstacle Challenge Map...
  Running A* Manhattan on Dynamic Obstacle Challenge Map...
```

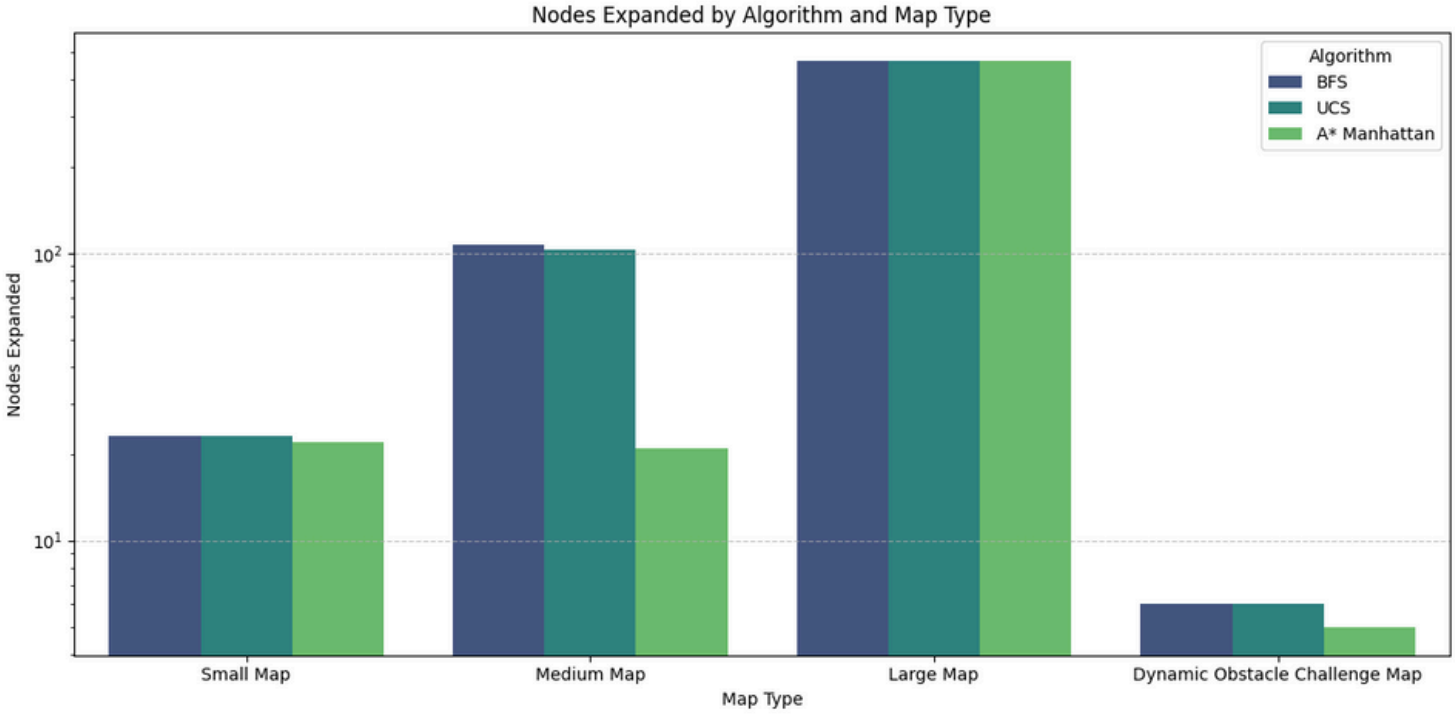
```
--- Running Dynamic Replanning Simulation (Proof-of-Concept) ---
Initial path planned: [(0, 0), (0, 1), (0, 2), (1, 2)] with cost 3.00

>>> Simulation Step 1 <<<

--- Updating Dynamic Obstacles ---
  Obstacle 0 moved to (0, 1)
!!! Collision detected at next step (0, 1)! Replanning needed.
--- Replanning Path ---
New path planned: [(0, 0), (1, 0), (1, 1), (1, 2)] with cost 3.00
Agent moved to (1, 0). Total cost: 1.00, Time: 1
Agent Status: Position=(1, 0), Cost=1.00, Time=1

>>> Simulation Step 2 <<<

--- Updating Dynamic Obstacles ---
  Obstacle 0 moved to (0, 1)
Agent moved to (1, 1). Total cost: 2.00, Time: 2
Agent Status: Position=(1, 1), Cost=2.00, Time=2

>>> Simulation Step 3 <<<

--- Updating Dynamic Obstacles ---
  Obstacle 0 moved to (0, 1)
Agent moved to (1, 2). Total cost: 3.00, Time: 3
Agent reached the goal!
Agent Status: Position=(1, 2), Cost=3.00, Time=3

--- Dynamic Replanning Simulation Summary ---
Final Agent Position: (1, 2)
Total Cost: 3.00
Total Time Steps: 3
Goal Reached: True

All simulations and experiments completed.
```

--- Experiment Results ---

algorithm	start_pos	goal_pos	path_found	path_length	path_cost	execution_time_ms	nodes_expanded	map_name	map_width	map_height
BFS	(0, 0)	(4, 4)	True	9	8	0.094595	23	Small Map	5	5
UCS	(0, 0)	(4, 4)	True	9	8	0.106895	23	Small Map	5	5
A* Manhattan	(0, 0)	(4, 4)	True	9	8	0.1051	22	Small Map	5	5
BFS	(6, 7)	(5, 13)	True	10	9	0.347142	107	Medium Map	15	15
UCS	(6, 7)	(5, 13)	True	10	9	0.523372	103	Medium Map	15	15
A* Manhattan	(6, 7)	(5, 13)	True	10	9	0.116129	21	Medium Map	15	15
BFS	(24, 15)	(18, 0)	False	0	inf	1.80804	465	Large Map	25	25
UCS	(24, 15)	(18, 0)	False	0	inf	3.33924	465	Large Map	25	25
A* Manhattan	(24, 15)	(18, 0)	False	0	inf	3.23899	465	Large Map	25	25
BFS	(0, 0)	(1, 2)	True	4	3	0.048572	6	Dynamic Obstacle Challenge Map	3	3
UCS	(0, 0)	(1, 2)	True	4	3	0.026568	6	Dynamic Obstacle Challenge Map	3	3
A* Manhattan	(0, 0)	(1, 2)	True	4	3	0.019611	5	Dynamic Obstacle Challenge Map	3	3

SCREENSHOTS AND RESULTS





TEST APPROACH

- Running experiments: We throw different maps and problems at each pathfinding method (BFS, UCS, A*) to see how they perform.
- Checking results: We verify if paths are found, what their cost is, and how long it took.
- Monitoring efficiency: We count how much 'thinking' (nodes expanded) each method does.
- Testing dynamic scenarios: We specifically test if the agent can adapt and find new paths when obstacles move and block its way.

CHALLENGES FACED

- Dynamic Updates
- Algorithm Correctness
- Environment Set-Up
- Importing Maps and Linking them to main.py

LEARNING AND KEY TAKEAWAYS

- End to end ML Work Flow.
- Getting familiar with pathfinding algorithms.
- Ability to handle multiple complexities.

FUTURE ENHANCEMENTS

- More Search Methods: Adding even more ways to find paths.
- Smarter Obstacles: Making the moving obstacles behave more cleverly.
- Live Map View: Creating a visual display to watch the agent and obstacles move in real-time.
- Complex Environments: Handling maps that are 3D or have more varied terrains.
- Multiple Agents: Allowing several agents to navigate the map together, avoiding collisions.

REFRENCES

- Pandas Documentation
- Google Collab
- Git-hub

THANK YOU